

String contra OpenJDK

La prueba

El fuzzer generó expresiones de `java.lang.String` con valores elegidos hacia los bordes, y las corrió en **Eclipse Adoptium 25.0.4.1** y en **KajiJDK con KajiLibrary**. O sea, se llamó a dos clases que deberían dar el mismo resultado, el `String` de OpenJDK y el de KajiLibrary, y se las probó con entradas aleatorias. Después comparó los resultados uno por uno y redujo cada divergencia distinta a su caso más chico.

Campaña terminada

Referencia **Eclipse Adoptium 25.0.4.1** Bajo prueba **KajiJDK (run-headless) con KajiLibrary**

Semilla **1** · 6 lotes de 120 17 de septiembre de 2026 a las 2:55 p. m.

720 expresiones comparadas	715 dieron lo mismo	5 divergieron	2 errores distintos en KajiLibrary
3 casos mínimos	23 s de campaña		

otro valor: 5

Qué es una divergencia

Una **divergencia** es una expresión en la que OpenJDK y KajiJDK devolvieron resultados distintos. Por ejemplo, `String.valueOf(s(56283)).compareToIgnoreCase(s(56274, 56703)))` da `-1011108` en OpenJDK y `9` en KajiJDK. Como OpenJDK hace de referencia, cada divergencia indica que KajiLibrary se aparta de lo que tendría que hacer.

Las etiquetas de arriba dicen de qué forma difieren:

- **otro valor**: las dos devolvieron un resultado, pero distinto.

Una divergencia no es lo mismo que un error: varias pueden salir del mismo error, con entradas distintas. Las 5 divergencias de esta campaña se explican con 2 errores, que se detallan abajo.

Qué encontró

El reductor achicó cada divergencia distinta hasta su caso mínimo, y varias quedaron en la misma expresión. Acá están agrupadas por causa. Los números salen de la campaña; la explicación de cada causa se escribió a mano, después de leer el código de las dos implementaciones y verificarla en OpenJDK.

Comparar ignorando mayúsculas solo funciona con la A a la Z

Error de KajiLibrary

KajiLibrary pliega cada carácter con una función que suma 32 cuando está entre A y Z, y deja todo lo demás igual. OpenJDK pliega por punto de código, con las mayúsculas y minúsculas de Unicode y tratando los pares de surrogates. Así, para OpenJDK el signo ohm Ω (U+2126) se pliega a w y para KajiLibrary no se pliega, y la comparación devuelve otro número. El mismo plegado lo usan equalsIgnoreCase y el comparador CASE_INSENSITIVE_ORDER, así que el error los alcanza a los tres.

KajiLibrary: java/lang/String.java, fold, líneas 766 a 771. OpenJDK: StringUTF16.compareToCIImpl y StringLatin1.compareToCI.

```
String.valueOf(s(56283).compareToIgnoreCase(s(56274, 56703)))
```

OPENJDK	KAJIJDK
-1011108	9

Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 87 a 61 caracteres en 2 pasos

```
String.valueOf(s(8486).compareToIgnoreCase(s(55297)))
```

OPENJDK	KAJIJDK
-54328	-46811

Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 197 a 53 caracteres en 9 pasos

Buscar una cadena vacía más allá del final devuelve -1 Error de KajiLibrary

La especificación de `indexOf(String, int)` dice que la respuesta arranca en `Math.min(fromIndex, this.length())`, y que una cadena vacía siempre se encuentra: `"".indexOf("", 1)` da 0. OpenJDK acota `fromIndex` al largo de la cadena y, si lo buscado está vacío, devuelve esa posición. KajiLibrary delega en su propia versión de tres argumentos, que tiene otro contrato —hasta dónde buscar—, y ahí una posición de arranque mayor que el final significa que no hay nada que encontrar: devuelve -1.

KajiLibrary: `java/lang/String.java`, `indexOf(String, int)` en la línea 894, que llama a `indexOf(String, int, int)` en la 904.
OpenJDK: `String.indexOf(byte[], byte, int, String, int)`, que hace `Math.clamp(fromIndex, 0, srcCount)`.

```
String.valueOf(s().indexOf(s(), 1))
```

OPENJDK

0

KAJIJDk

-1

Una divergencia de la campaña se redujo a esta expresión.

Cómo se redujo: de 47 a 35 caracteres en 2 pasos

Dónde se concentran las divergencias

Para cada operación, en cuántas de las expresiones que la usan hubo divergencia. Que una operación aparezca en una expresión que diverge no prueba que sea la culpable: eso lo dicen los casos mínimos.



Todas las divergencias

Cada expresión en la que OpenJDK y KajiJDK dieron un resultado distinto, en el orden en que aparecieron.

#	TIPO	EXPRESIÓN GENERADA	OPENJDK	KAJIJDK
1	otro valor	String.valueOf(s(32, 8486, 122, 56283, 47, 49337)).compareToIgnoreCase(s(56274, 56703))	-1067359	-56242
2	otro valor	String.valueOf(s(41116, 52566).indexOf(s(), 3))	2	-1

#	TIPO	EXPRESIÓN GENERADA	OPENJDK	KAJIJDK
3	otro valor	<code>String.valueOf(s(82, 304, 769, 67, 92, 8486, 42, 8486).indexOf(s(), 100))</code>	8	-1
4	otro valor	<code>String.valueOf(s(56272, 57206, 56320, 106, 111).compareToIgnoreCase(s(56114, 56773, 55296, 55620, 57182, 39, 769, 931, 56067, 56641, 453, 7838)))</code>	162225	158
5	otro valor	<code>String.valueOf(s(101, 8486, 10, 56, 223, 65279, 453, 55596, 57077, 90, 160, 16420).replace((char) 223, (char) 65).compareToIgnoreCase(s(55297, 56320, 7838, 55357, 56832, 962, 33, 111, 78).strip()))</code>	-66499	-55196

Cómo se obtuvo

- **Generar.** 6 lotes de 120 expresiones de `String`, de hasta tres niveles de operaciones anidadas. Los caracteres salen, casi la mitad de las veces, de una lista de bordes: espacios raros, la ß alemana, la I y la i turcas, la sigma final griega, marcas combinables, surrogates sueltos y emojis de fuera del plano básico.
- **Ejecutar.** Cada lote es un único programa Java, compilado una sola vez con el `javac` de Eclipse Adoptium 25.0.4.1. El mismo `.class` corre en ese JDK y en KajiJDK con KajiLibrary.
- **Comparar.** Dos resultados coinciden si su texto es idéntico. Cuando una expresión lanza una excepción se compara la clase de la excepción, no el mensaje, que puede variar entre implementaciones sin estar mal.
- **Reducir.** De cada tipo de divergencia distinto se toma la primera, y se prueban versiones más chicas: quitar una operación, cambiar una constante por 0, 1, -1 o 2, acortar sus dígitos. Se queda con la más chica que todavía diverge del mismo modo, y repite.
- **Reproducir.** La misma semilla genera las mismas expresiones: `python test-fuzzing/fuzzer/strfuzz.py --jdk "$OPENJDK" --seed 1 --lotes 6 --por-lote 120`.

Dónde sirve

- **Reescrituras y migraciones.** La versión vieja hace de oráculo, así que no hay que escribir el resultado esperado de cada caso.
- **Versión contra versión.** La release de hoy contra la de ayer, sobre las mismas entradas generadas.
- **Código numérico.** Redondeos, desbordes, divisiones, fechas y coma flotante: donde viven los bordes.
- **Parsers y serializadores.** Lo que se codifica y se decodifica tiene que volver igual.
- **Dos implementaciones de una misma especificación.** Cualquier diferencia es un error de

Lo que no resuelve

- **Necesita una referencia.** Una implementación confiable, o una propiedad que valga siempre, como que decodificar lo codificado devuelva la entrada.
- **Necesita determinismo.** Fecha, hora y todo lo aleatorio tienen que quedar fijos, o el oráculo reporta ruido.
- **Encuentra diferencias, no culpables.** Qué versión tiene razón lo decide una persona con la especificación en la mano.
- **Hay que auditar el instrumento.** Un oráculo roto, por ejemplo uno que compare una implementación

una de las dos, como KajiLibrary y OpenJDK en este informe.

consigo misma, reporta cero divergencias aunque haya errores.

Datos de 17 de septiembre de 2026 a las 2:55 p. m.. KajiJDK compilado el 17 de septiembre de 2026 a las 11:59 a. m.. Tiempo de ejecución: 1 s en OpenJDK y 1 s en KajiJDK, sin contar la reducción.